Week 6 - Bivariate Analysis, part 2

1. Lesson: no lesson this week

2. Weekly graph question

Suppose you wanted to show the nitrate and phosphate level in a water sample. What are the advantages and disadvantages of showing this as an area plot, as opposed to two separate line graphs? How would you adjust the graphs shown to improve the presentation?

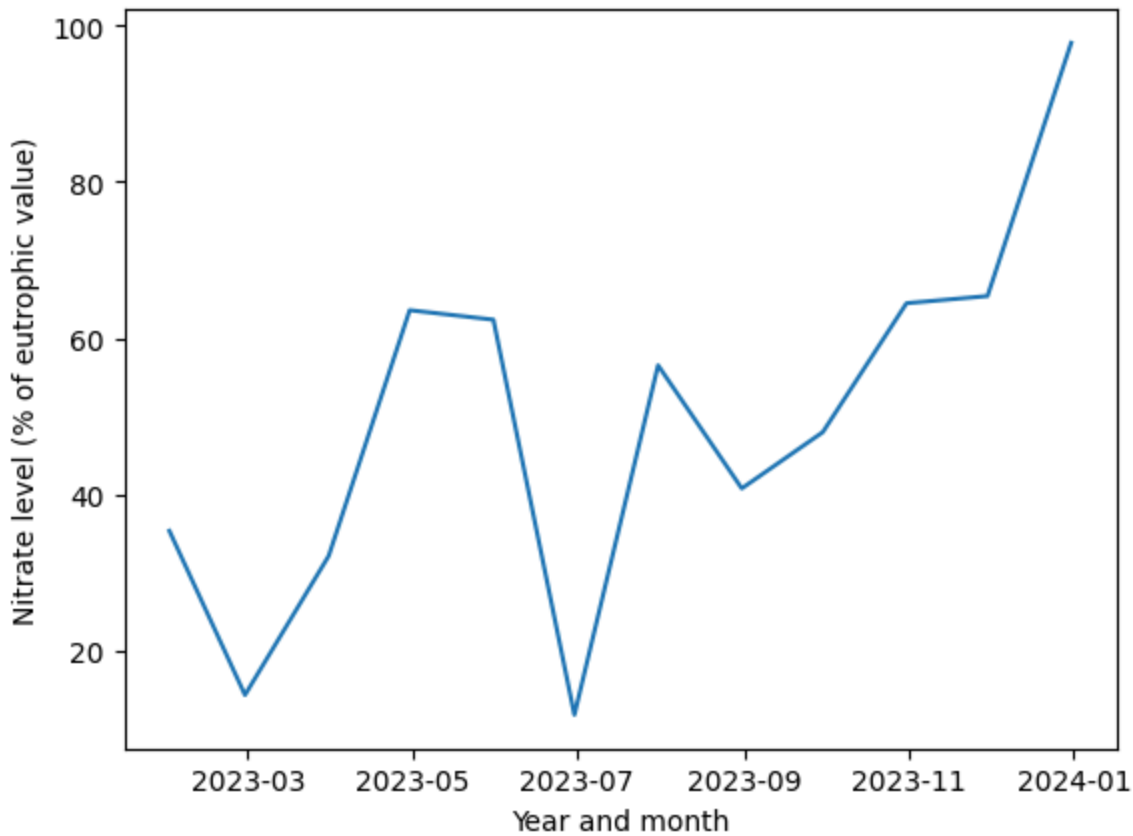
Although the area plot perserves the first plot's data, the second plot's visuals are not as clear. The y-axis, which represented percents previously, is now showing values over 100% for the second plot since it considers 0 to be where the first plot is. This makes it difficult to visualize the percentages of the second plot. One improvement would be to add labels to each individual point on each line.

In [1]:
```
import numpy as np
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
```

In [3]:
```
numdates = 12
np.random.seed(0)
time_series = 100 * (np.random.normal(size = numdates) / 5 + np.arange(numdates) /
datearray = pd.date_range(start='2023/01/01', end='2024/01/01', freq='M')
ax = sns.lineplot(x = datearray, y = time_series)
ax.set_xlabel("Year and month")
ax.set_ylabel("Nitrate level (% of eutrophic value)")
```

```
C:\Users\caleb\AppData\Local\Temp\ipykernel_5200\2149468501.py:4: FutureWarning: 'M'
is deprecated and will be removed in a future version, please use 'ME' instead.
  datearray = pd.date_range(start='2023/01/01', end='2024/01/01', freq='M')
```

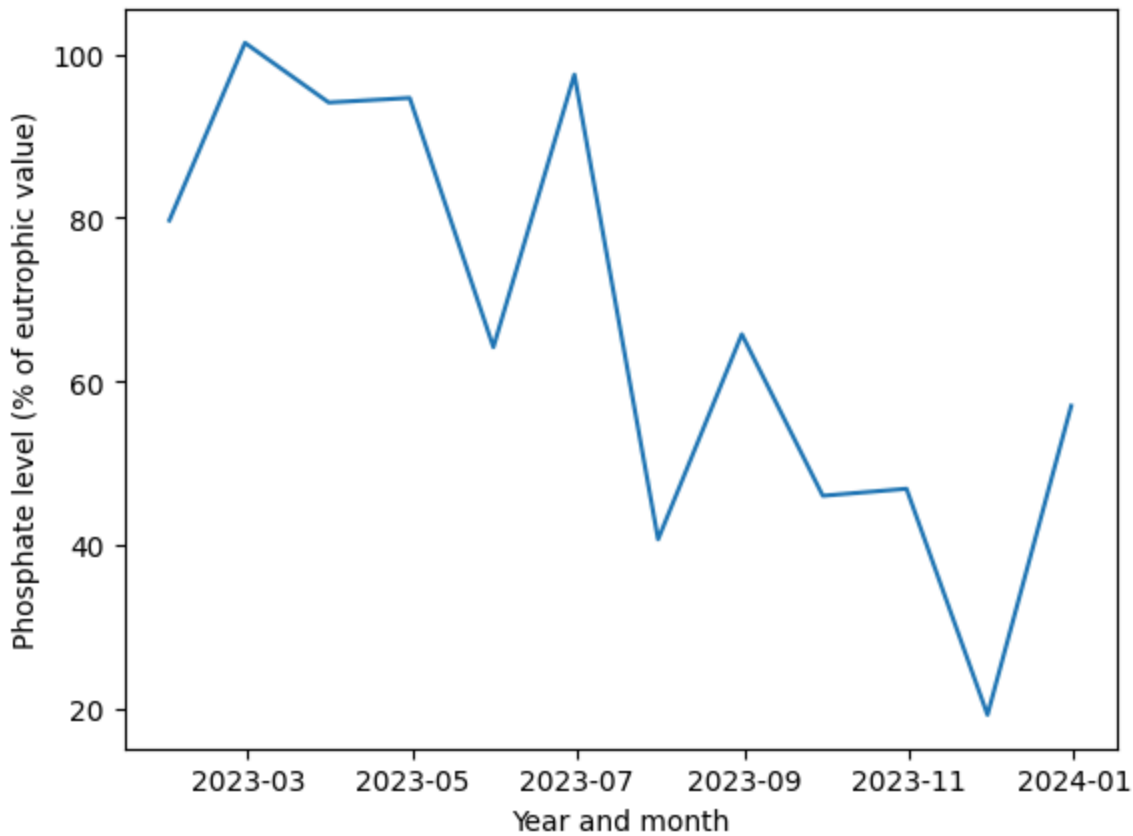
Out[3]:
```
Text(0, 0.5, 'Nitrate level (% of eutrophic value)')
```



In [4]:
```
np.random.seed(1)
time_series_2 =  100 * (1 - (np.random.normal(size = numdates) / 8 + np.arange(numd
datearray = pd.date_range(start='2023/01/01', end='2024/01/01', freq='M')
ax = sns.lineplot(x = datearray, y = time_series_2)
ax.set_xlabel("Year and month")
ax.set_ylabel("Phosphate level (% of eutrophic value)")
```

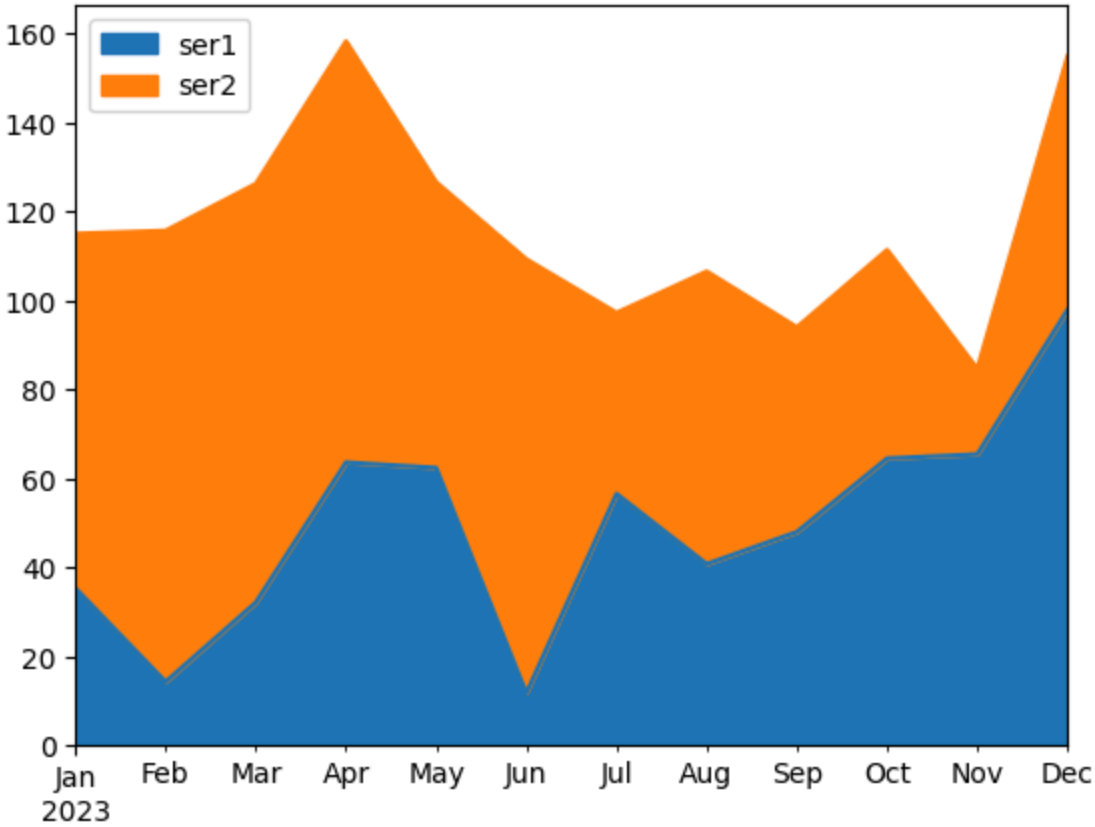
```
C:\Users\caleb\AppData\Local\Temp\ipykernel_5200\1726504607.py:3: FutureWarning: 'M'
is deprecated and will be removed in a future version, please use 'ME' instead.
  datearray = pd.date_range(start='2023/01/01', end='2024/01/01', freq='M')
```

Out[4]:
```
Text(0, 0.5, 'Phosphate level (% of eutrophic value)')
```



In [5]:
```
df = pd.DataFrame({'ser1': time_series, 'ser2': time_series_2}, index = datearray)
df.plot.area()
```

Out[5]: <Axes: >



3. Working on your datasets

This week, you will do the same types of exercises as last week, but you should use your chosen datasets that someone in your class found last semester. (They likely will not be the particular datasets that you found yourself.)

Here are some types of analysis you can do:

- Find correlations between pairs of variables.
- Draw scatterplots, especially when the correlation is large.
- Draw pairplots.
- Draw line graphs and/or area graphs when there is date or time data together with numerical data.

Conclusions:

- Explain what conclusions you would draw from this analysis: are the data what you expect? Are the data likely to be usable? If the data are not useable, find some new data!
- Do you see any outliers? (Data points that are far from the rest of the data).

- Are any data items highly correlated with each other, suggesting that they are redundant?
- For the line plots, do you see a trend or pattern over time? Does this suggest that the data are changing over time (drifting) in such a way as to invalidate comparisons?
- Can you think of any confounding variables? (Third variables that could explain any correlations between other variables. These third variables may or may not be reported in the dataset.)

Read Data

In [6]:
```
# https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laund
li_small_accounts_df = pd.read_csv('C:/Users/caleb/Downloads/ibm_aml/LI-Small_accou
li_small_accounts_df
```

Out[6]:

	Bank Name	Bank ID	Account Number	Entity ID	Entity Name
0	China Bank #2820	314693	81B86A280	800D8CCF0	Corporation #41344
1	France Bank #4585	311253	8187FEA80	800B505E0	Corporation #54497
2	China Bank #2242	39996	803961E00	800D03F60	Partnership #36904
3	National Bank of Newport	331440	81B075800	801567C10	Corporation #16224
4	UK Bank #33	135417	80CF87C80	801085E00	Partnership #72930
...	...	...	...	...	...
712683	China Bank #42	692	80346A5F0	800D080A0	Corporation #40025
712684	First Bank of Watertown	118699	80B802A70	8005319C0	Partnership #14803
712685	Bank of Lincoln	213123	80847BE70	800453480	Partnership #11893
712686	Hilltop Credit Union	18747	803A36270	8003B5120	Partnership #6726
712687	Savings Bank of Helena	25242	80B66B2F0	8003122E0	Corporation #15281

712688 rows × 5 columns

In [7]:
```
li_small_trans_df = pd.read_csv('C:/Users/caleb/Downloads/ibm_aml/LI-Small_trans.cs
li_small_trans_df
```

Out[7]:

	Timestamp	From Bank	Account	To Bank	Account.1	Amount Received	Receiving Currency	A
0	2022/09/01 00:08	11	8000ECA90	11	8000ECA90	3.195403e+06	US Dollar	3
1	2022/09/01 00:21	3402	80021DAD0	3402	80021DAD0	1.858960e+03	US Dollar	1
2	2022/09/01 00:00	11	8000ECA90	1120	8006AA910	5.925710e+05	US Dollar	5
3	2022/09/01 00:16	3814	8006AD080	3814	8006AD080	1.232000e+01	US Dollar	1
4	2022/09/01 00:00	20	8006AD530	20	8006AD530	2.941560e+03	US Dollar	2
...	...	...	...	...	...	...	...	
6924044	2022/09/10 23:39	71696	81B2518F1	71528	81C0482E1	3.346900e-02	Bitcoin	
6924045	2022/09/10 23:48	271241	81B567481	173457	81C0DA751	1.313000e-03	Bitcoin	
6924046	2022/09/10 23:50	271241	81B567481	173457	81C0DA751	1.305800e-02	Bitcoin	
6924047	2022/09/10 23:57	170558	81A2206B1	275798	81C1D5CA1	4.145370e-01	Bitcoin	
6924048	2022/09/10 23:31	170558	81A2206B1	275798	81C1D5CA1	3.427700e-02	Bitcoin	

6924049 rows × 11 columns

In [8]:
```
hi_small_accounts_df = pd.read_csv('C:/Users/caleb/Downloads/ibm_aml/HI-Small_accou
hi_small_accounts_df
```

Out[8]:

	Bank Name	Bank ID	Account Number	Entity ID	Entity Name
0	Portugal Bank #4507	331579	80B779D80	80062E240	Sole Proprietorship #50438
1	Canada Bank #27	210	809D86900	800C998A0	Corporation #33520
2	UK Bank #33	21884	80812BE00	800C47F50	Partnership #35397
3	Germany Bank #4815	32742	81047F300	80096F0B0	Corporation #48813
4	National Bank of Harrisburg	127390	80BD8CF00	800FB8760	Corporation #889
...	...	...	...	...	...
518576	France Bank #33	3881	807886B70	80062D160	Sole Proprietorship #22257
518577	National Bank of Topeka	333423	81314C870	800F40230	Sole Proprietorship #4995
518578	Plandor Trust Bank	1467	804ED2270	800CAC4C0	Sole Proprietorship #35326
518579	Sappo Bancorp	2843	801727270	800EFCB40	Sole Proprietorship #3522
518580	Savings Bank of Montpelier	213580	804F6D9F0	800406D60	Partnership #49800

518581 rows × 5 columns

In [9]:
```
hi_small_trans_df = pd.read_csv('C:/Users/caleb/Downloads/ibm_aml/HI-Small_trans.cs
hi_small_trans_df
```

Out[9]:

	Timestamp	From Bank	Account	To Bank	Account.1	Amount Received	Receiving Currency	Ar
0	2022/09/01 00:20	10	8000EBD30	10	8000EBD30	3697.340000	US Dollar	3
1	2022/09/01 00:20	3208	8000F4580	1	8000F5340	0.010000	US Dollar	
2	2022/09/01 00:00	3209	8000F4670	3209	8000F4670	14675.570000	US Dollar	14
3	2022/09/01 00:02	12	8000F5030	12	8000F5030	2806.970000	US Dollar	2
4	2022/09/01 00:06	10	8000F5200	10	8000F5200	36682.970000	US Dollar	36
...	...	...	...	...	...	...	...	
5078340	2022/09/10 23:57	54219	8148A6631	256398	8148A8711	0.154978	Bitcoin	
5078341	2022/09/10 23:35	15	8148A8671	256398	8148A8711	0.108128	Bitcoin	
5078342	2022/09/10 23:52	154365	8148A6771	256398	8148A8711	0.004988	Bitcoin	
5078343	2022/09/10 23:46	256398	8148A6311	256398	8148A8711	0.038417	Bitcoin	
5078344	2022/09/10 23:37	154518	8148A6091	256398	8148A8711	0.281983	Bitcoin	

5078345 rows × 11 columns

Summarize Data

In [10]:
```
li_small_accounts_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 712688 entries, 0 to 712687
Data columns (total 5 columns):
 #   Column          Non-Null Count   Dtype 
---  ------          --------------   ----- 
 0   Bank Name       712688 non-null  object
 1   Bank ID         712688 non-null  int64 
 2   Account Number  712688 non-null  object
 3   Entity ID       712688 non-null  object
 4   Entity Name     712688 non-null  object
dtypes: int64(1), object(4)
memory usage: 27.2+ MB
```

In [11]:
```
li_small_trans_df.info(show_counts=True)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6924049 entries, 0 to 6924048
Data columns (total 11 columns):
 #   Column              Non-Null Count    Dtype
---  ------              --------------    -----
 0   Timestamp           6924049 non-null  object
 1   From Bank           6924049 non-null  int64
 2   Account             6924049 non-null  object
 3   To Bank             6924049 non-null  int64
 4   Account.1           6924049 non-null  object
 5   Amount Received     6924049 non-null  float64
 6   Receiving Currency  6924049 non-null  object
 7   Amount Paid         6924049 non-null  float64
 8   Payment Currency    6924049 non-null  object
 9   Payment Format      6924049 non-null  object
 10  Is Laundering       6924049 non-null  int64
dtypes: float64(2), int64(3), object(6)
memory usage: 581.1+ MB
```

In [12]:
```
hi_small_accounts_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 518581 entries, 0 to 518580
Data columns (total 5 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   Bank Name       518581 non-null  object
 1   Bank ID         518581 non-null  int64
 2   Account Number  518581 non-null  object
 3   Entity ID       518581 non-null  object
 4   Entity Name     518581 non-null  object
dtypes: int64(1), object(4)
memory usage: 19.8+ MB
```

In [13]:
```
hi_small_trans_df.info(show_counts=True)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5078345 entries, 0 to 5078344
Data columns (total 11 columns):
 #   Column              Non-Null Count    Dtype
---  ------              --------------    -----
 0   Timestamp           5078345 non-null  object
 1   From Bank           5078345 non-null  int64
 2   Account             5078345 non-null  object
 3   To Bank             5078345 non-null  int64
 4   Account.1           5078345 non-null  object
 5   Amount Received     5078345 non-null  float64
 6   Receiving Currency  5078345 non-null  object
 7   Amount Paid         5078345 non-null  float64
 8   Payment Currency    5078345 non-null  object
 9   Payment Format      5078345 non-null  object
 10  Is Laundering       5078345 non-null  int64
dtypes: float64(2), int64(3), object(6)
memory usage: 426.2+ MB
```

There are no null values in any of the datasets

The account dataframes show details about accounts, what bank the account belongs to, and the business entity that owns the bank account

The transactions show different types of payments between accounts

The reasoning for the datasets being split into two groups is as follows:

- Group HI has a relatively higher illicit ratio (more laundering).
- Group LI has a relatively lower illicit ratio (less laundering).

Check for Inifinite Values and Duplicates

In [14]:
```
hi_small_accounts_num_df = hi_small_accounts_df.select_dtypes(include=[np.number])

# Find rows where ANY column has an infinite value
rows_with_inf = hi_small_accounts_df[np.isinf(hi_small_accounts_num_df).any(axis=1)
rows_with_inf
```

Out[14]:

Bank Name	Bank ID	Account Number	Entity ID	Entity Name

In [15]:
```
li_small_accounts_num_df = li_small_accounts_df.select_dtypes(include=[np.number])
# Find rows where ANY column has an infinite value
rows_with_inf = li_small_accounts_df[np.isinf(li_small_accounts_num_df).any(axis=1)
rows_with_inf
```

Out[15]:

Bank Name	Bank ID	Account Number	Entity ID	Entity Name

In [16]:
```
hi_small_trans_num_df = hi_small_trans_df.select_dtypes(include=[np.number])

# Find rows where ANY column has an infinite value
rows_with_inf = hi_small_trans_df[np.isinf(hi_small_trans_num_df).any(axis=1)]
rows_with_inf
```

Out[16]:

Timestamp	From Bank	Account	To Bank	Account.1	Amount Received	Receiving Currency	Amount Paid	Payment Currency	I

In [17]:
```
li_small_trans_num_df = li_small_trans_df.select_dtypes(include=[np.number])

# Find rows where ANY column has an infinite value
rows_with_inf = li_small_trans_df[np.isinf(li_small_trans_num_df).any(axis=1)]
rows_with_inf
```

Out[17]:

Timestamp	From Bank	Account	To Bank	Account.1	Amount Received	Receiving Currency	Amount Paid	Payment Currency	I

There are no inf values

In [18]:
```
hi_small_accounts_df[hi_small_accounts_df.duplicated(keep=False)]
```

Out[18]:

Bank Name	Bank ID	Account Number	Entity ID	Entity Name

In [19]:
```
li_small_accounts_df[li_small_accounts_df.duplicated(keep=False)]
```

Out[19]:

Bank Name	Bank ID	Account Number	Entity ID	Entity Name

In [20]:
```
hi_small_trans_df[hi_small_trans_df.duplicated(keep=False)]
```

Out[20]:

	Timestamp	From Bank	Account	To Bank	Account.1	Amount Received	Receiving Currency	Amou Pa
863293	2022/09/01 16:20	12004	800C927C1	12004	800C927C0	0.000008	Bitcoin	0.0800
863294	2022/09/01 16:20	12004	800C927C1	220	813D8C1E1	0.000008	Bitcoin	0.0000
863295	2022/09/01 16:20	12004	800C927C1	12004	800C927C0	0.000008	Bitcoin	0.0800
863296	2022/09/01 16:20	12004	800C927C1	220	813D8C1E1	0.000008	Bitcoin	0.0000
3684003	2022/09/07 21:25	29992	8099A29B1	220	813725AE1	0.000003	Bitcoin	0.0000
3684005	2022/09/07 21:25	29992	8099A29B1	220	813725AE1	0.000003	Bitcoin	0.0000
4166784	2022/09/08 21:05	113779	811144AB1	113779	811144AB0	0.000002	Bitcoin	0.0200
4166785	2022/09/08 21:05	113779	811144AB1	53744	813C777F1	0.000002	Bitcoin	0.0000
4166786	2022/09/08 21:05	113779	811144AB1	113779	811144AB0	0.000002	Bitcoin	0.0200
4166787	2022/09/08 21:05	113779	811144AB1	53744	813C777F1	0.000002	Bitcoin	0.0000
4510478	2022/09/09 10:03	6075	80C702911	6075	80C702910	0.000002	Bitcoin	0.0200
4510479	2022/09/09 10:03	6075	80C702911	154653	814389B61	0.000002	Bitcoin	0.0000
4510480	2022/09/09 10:03	6075	80C702911	6075	80C702910	0.000002	Bitcoin	0.0200
4510481	2022/09/09 10:03	6075	80C702911	154653	814389B61	0.000002	Bitcoin	0.0000
4816510	2022/09/09 21:33	14433	80935A891	14433	80935A890	0.000001	Bitcoin	0.0100
4816511	2022/09/09 21:33	14433	80935A891	15	813F7AE61	0.000001	Bitcoin	0.0000
4816512	2022/09/09 21:33	14433	80935A891	14433	80935A890	0.000001	Bitcoin	0.0100
4816513	2022/09/09 21:33	14433	80935A891	15	813F7AE61	0.000001	Bitcoin	0.0000

In [21]:
```
li_small_trans_df[li_small_trans_df.duplicated(keep=False)]
```

Out[21]:

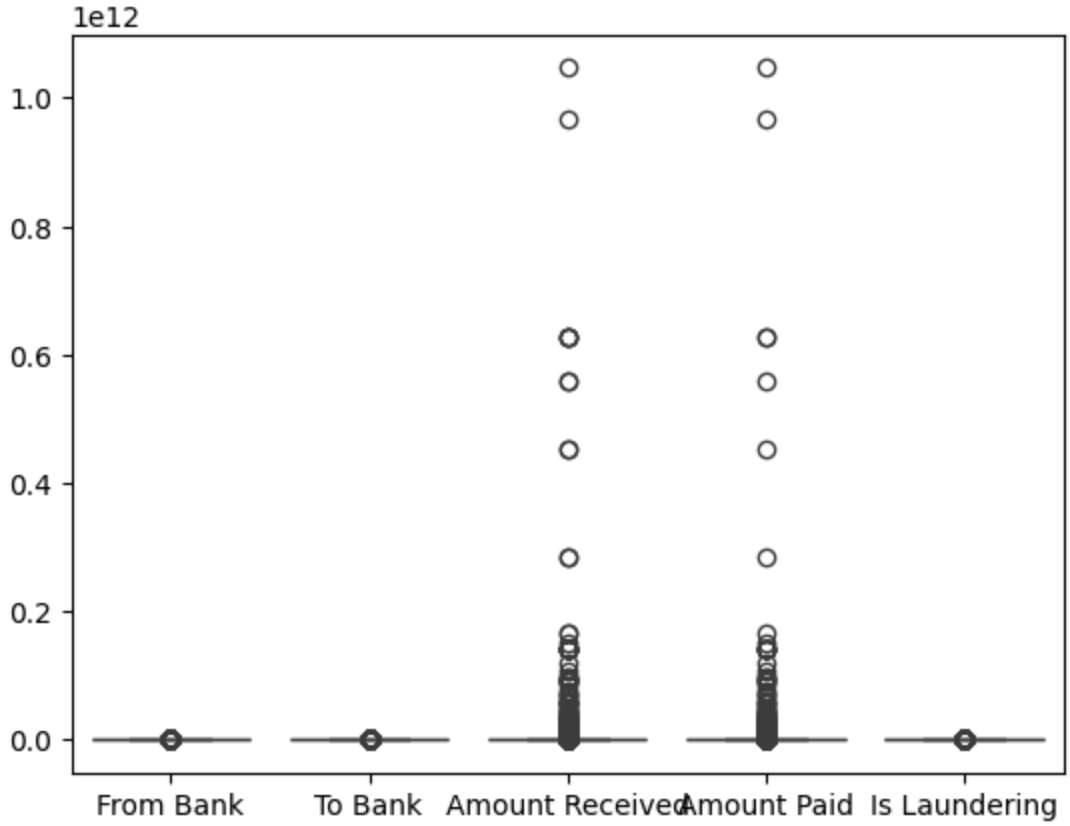
	Timestamp	From Bank	Account	To Bank	Account.1	Amount Received	Receiving Currency	Amou Pa
676694	2022/09/01 05:03	21260	8085A09D1	72146	81AB967E1	0.000001	Bitcoin	0.0000
676696	2022/09/01 05:03	21260	8085A09D1	72146	81AB967E1	0.000001	Bitcoin	0.0000
790828	2022/09/01 07:44	231616	814E45511	171546	81B324AC1	0.000001	Bitcoin	0.0000
790830	2022/09/01 07:44	231616	814E45511	171546	81B324AC1	0.000001	Bitcoin	0.0000
2027614	2022/09/02 11:09	28335	804B468F1	171669	81BBCDAB1	0.000007	Bitcoin	0.0000
2027616	2022/09/02 11:09	28335	804B468F1	171669	81BBCDAB1	0.000007	Bitcoin	0.0000
3482551	2022/09/05 12:44	22661	80D016C81	269294	81A245E81	0.000005	Bitcoin	0.0000
3482553	2022/09/05 12:44	22661	80D016C81	269294	81A245E81	0.000005	Bitcoin	0.0000
4180129	2022/09/06 14:16	19970	804E44F11	169489	8197DC971	0.000001	Bitcoin	0.0000
4180131	2022/09/06 14:16	19970	804E44F11	169489	8197DC971	0.000001	Bitcoin	0.0000
4559108	2022/09/07 03:42	113016	80A4D8561	113016	80A4D8560	0.000001	Bitcoin	0.0100
4559109	2022/09/07 03:42	113016	80A4D8561	169261	81993FDE1	0.000001	Bitcoin	0.0000
4559112	2022/09/07 03:42	113016	80A4D8561	113016	80A4D8560	0.000001	Bitcoin	0.0100
4559113	2022/09/07 03:42	113016	80A4D8561	169261	81993FDE1	0.000001	Bitcoin	0.0000
6422480	2022/09/09 17:56	11701	8034B3501	273006	81B49ADC1	0.000001	Bitcoin	0.0000
6422482	2022/09/09 17:56	11701	8034B3501	273006	81B49ADC1	0.000001	Bitcoin	0.0000

The identified duplicates are duplicate transactions. Since this just means the same amount is sent between the same accounts, I will leave these as valid samples.

Boxplots

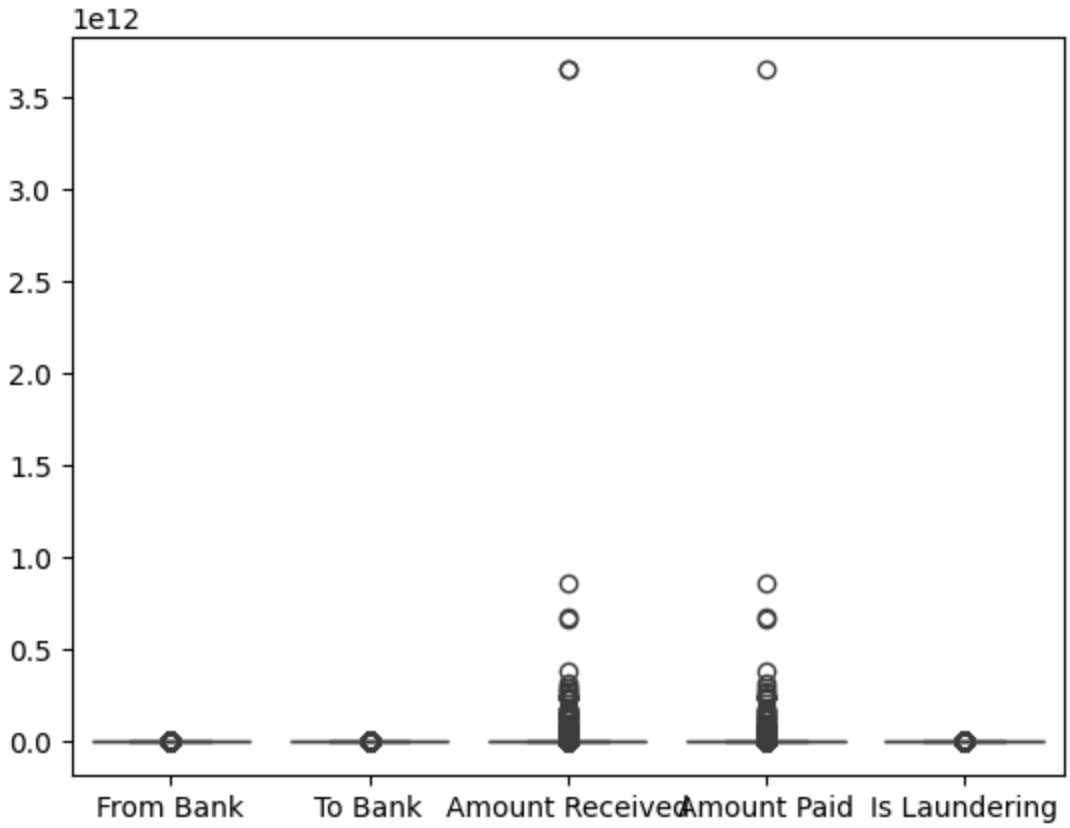
In [23]:
```
sns.boxplot(hi_small_trans_df)
```

Out[23]: <Axes: >



In [24]:
```
sns.boxplot(li_small_trans_df)
```

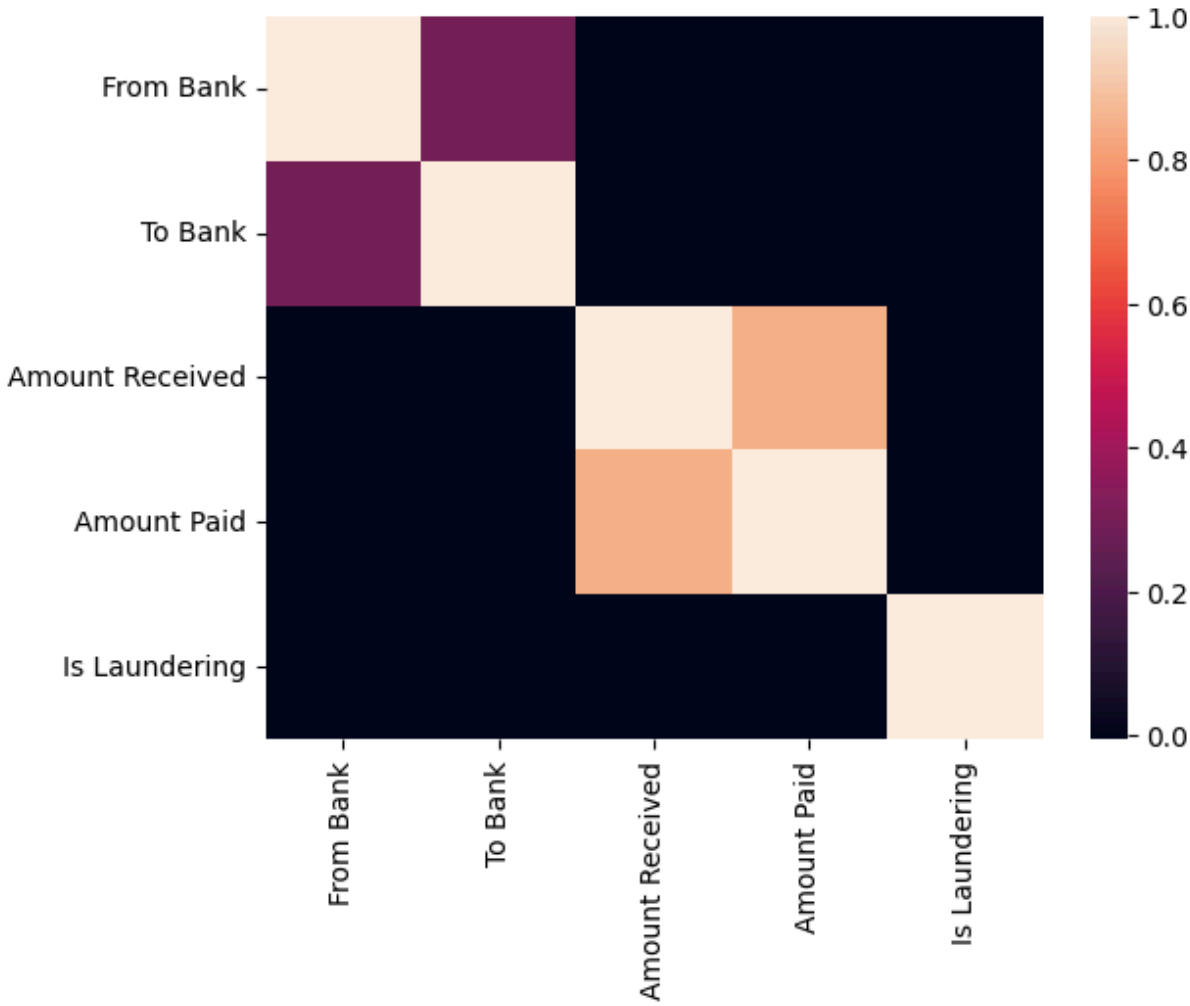
Out[24]: <Axes: >



Correlations

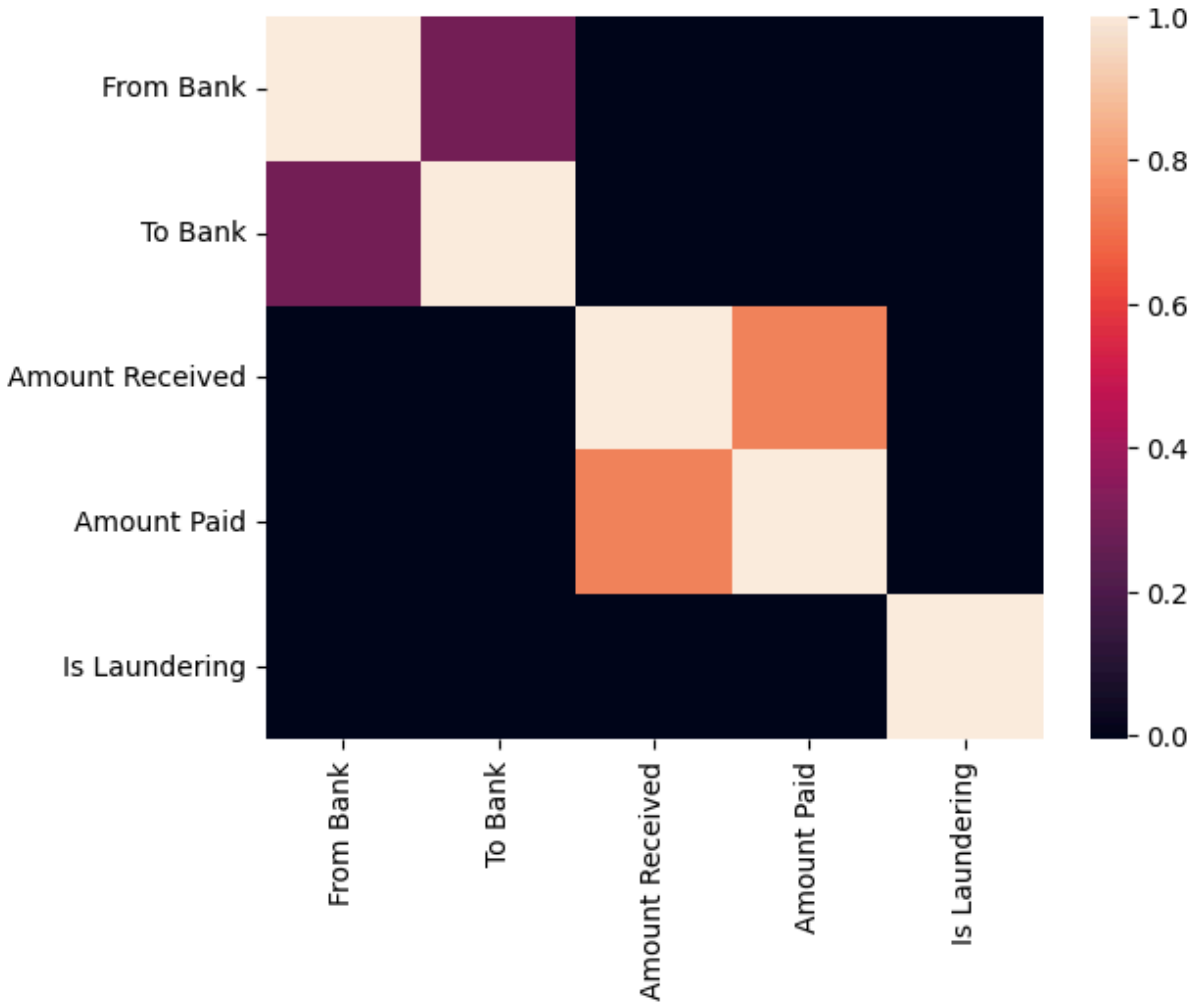
In [25]:
```
sns.heatmap(hi_small_trans_df.corr(numeric_only=True))
```

Out[25]: <Axes: >



In [26]:
```
sns.heatmap(li_small_trans_df.corr(numeric_only=True))
```

Out[26]: <Axes: >

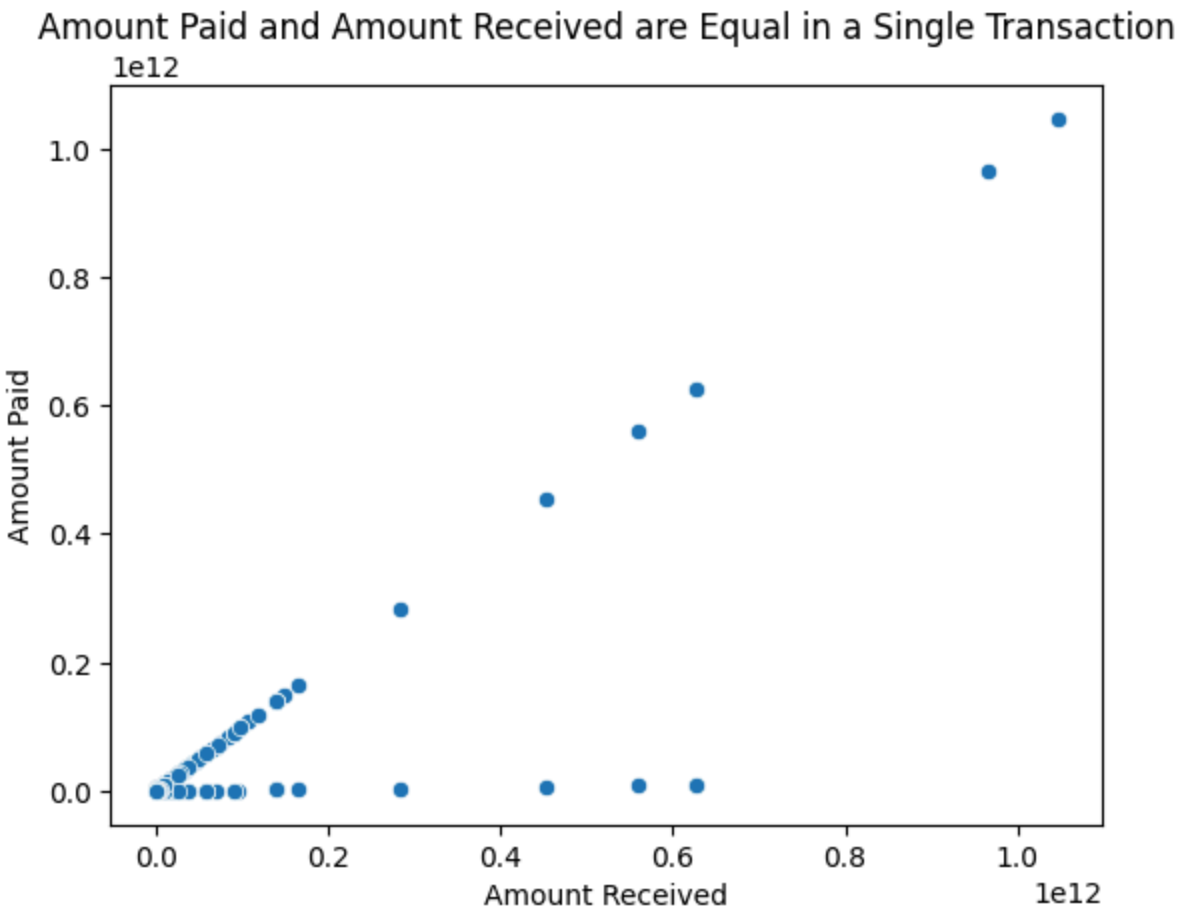


We see a strong correlation between Amount Received/Paid and To/From Bank. The other combinations do not have correlation.

The relationship between the Amounts is expected, since each transaction contains two accounts and the Received/Paid should mirror each other. We can see that in the scatterplot below.

I expect that the relationships in the data will reveal themselves not through just checking correlations, but by building a model of how transactions fit together. We need a full picture of how money flows throughout many transactions.

In [27]:
```
sns.scatterplot(x=hi_small_trans_df['Amount Received'], y=hi_small_trans_df['Amount
plt.title("Amount Paid and Amount Received are Equal in a Single Transaction")
None
```



Timeline of Transactions for a Sample Account

In [28]:
```
# Plotting a timeline of transactions that contains money laundering
hi_small_trans_df[hi_small_trans_df['Is Laundering'] == 1]
```

Out[28]:

	Timestamp	From Bank	Account	To Bank	Account.1	Amount Received	Receiving Currency	
4742	2022/09/01 00:21	70	100428660	1124	800825340	389769.39	US Dollar	3
40621	2022/09/01 00:03	70	100428660	11474	805B716C0	29024.33	US Dollar	
69152	2022/09/01 00:01	70	100428660	15980	80B39E7B0	792.92	US Dollar	
85763	2022/09/01 00:03	70	100428660	113798	80DC756E0	13171425.53	US Dollar	131
90684	2022/09/01 00:23	70	100428660	32375	80E480620	14288.83	US Dollar	
...	...	...	...	...	...	...	...	
5071168	2022/09/10 23:29	111433	8065C2380	21550	8065C69E0	7773.42	US Dollar	
5073375	2022/09/10 23:07	9	80727F930	219557	8072A28C0	448920.65	Ruble	4
5074248	2022/09/12 03:34	222	812D127D0	50202	812D129C0	20085.71	Saudi Riyal	
5075056	2022/09/10 23:43	17327	8095F6C20	15695	8095F6C70	3314.36	US Dollar	
5077204	2022/09/10 23:53	3	804FAA470	212789	804FAD580	3564699.78	Rupee	35

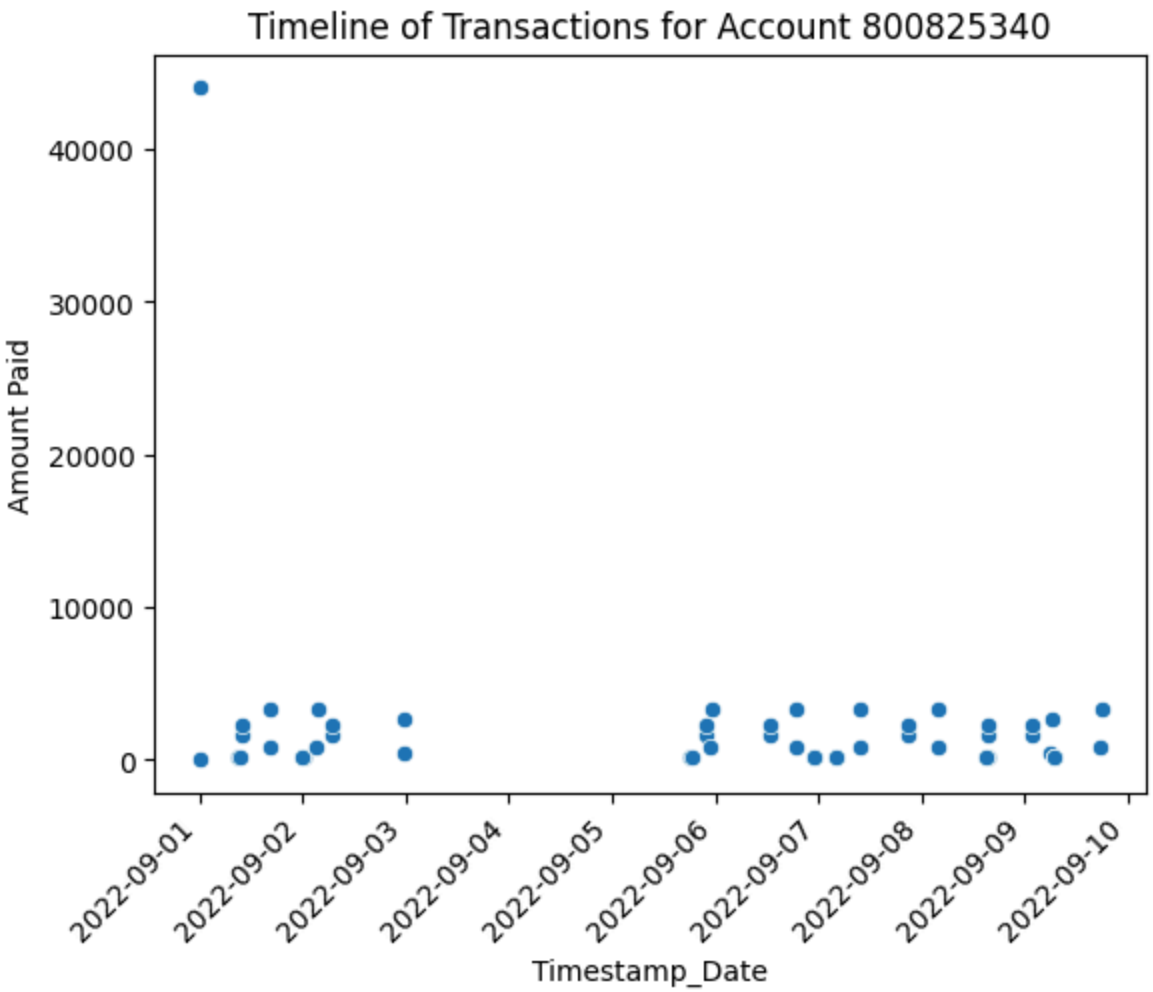
5177 rows × 11 columns

In [ ]:

```
hi_transactions_sample = hi_small_trans_df[hi_small_trans_df['Account'] == '8008253
hi_transactions_sample['Timestamp_Date'] = pd.to_datetime(hi_transactions_sample['T
sns.scatterplot(hi_transactions_sample, x="Timestamp_Date", y="Amount Paid")
plt.xticks(rotation=45, horizontalalignment='right')
plt.title("Timeline of Transactions for Account 800825340")
None
```

```
C:\Users\caleb\AppData\Local\Temp\ipykernel_13184\1523918599.py:2: SettingWithCopyWa
rning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/u
ser_guide/indexing.html#returning-a-view-versus-a-copy
  hi_transactions_sample['Timestamp_Date'] = pd.to_datetime(hi_transactions_sample
['Timestamp'])
```



Conclusions

The IBM synthetic accounts/transactions dataset is high quality. There are no missing values to handle. There are some duplicates instances, but none that we need to drop. Additionally, there are a large number of instances we can use for analysis.

In the boxplots, we can see some outliers for the Amount Recieved/Paid features. The outliers are for large amounts. The majority of transactions are for small sums

The Amount Received/Paid features are highly correlated. This is because they are equal and opposite to each other. Having the duplicate information will be useful for building the full relationship of transactions for a given account

Looking at a sample account which had fraudulent transaction, we see a number of smaller transactions and single large transaction. One strategy that a fraudster might utilize is spending a large sum from an account they have taken over. This does not cover all possible methods though. More in depth analysis will be needed to uncover these patterns.

The largest challenge with this dataset is that feature engineering is required before analysis. There is only a small number of features. The real insights into fraud will come from building

up a full picture of the transactions for each account. From here the model can learn what types of transactions are dissimilar to normal ones. One strategy I will employ is creating an engineered feature to represent the average purchase amount for an account, as well as the amount of times two accounts have sent money to each other.

4. Storytelling With Data plot

Reproduce any graph of your choice in p. 86-98 of the Storytelling With Data book as best you can. (The second half of chapter three). You do not have to get the exact data values right, just the overall look and feel.

In [116…

```
# Figure 3.16 (p. 88)
index = [
    'Selection',
    'Service',
    'Relationship',
    'Convenience',
    'Price'
]
storytelling_with_data_df = pd.DataFrame(
    {
        'Competitor E': [10,9,5,7,7],
        'Competitor D': [6,7,4.5,5,5],
        'Competitor C': [3,2,3.2,4.5,6],
        'Competitor B': [4,3,3,4,2],
        'Competitor A': [2.2,7,1,9,6],
        'Our Business': [2,2.4,3,8,8]
    }, index=index
)
ax = storytelling_with_data_df.plot.barh(color=['grey', 'grey', 'grey', 'grey', 'gr
plt.xticks([])
ax.set_frame_on(False)
plt.title("Performance Overview", loc='left')
plt.text(1, 4.4, 'Weighted performance index |', fontsize=10, color='grey')
plt.text(5.5, 4.4, 'relative rank', fontsize=10, color='royalblue')
plt.text(8.1, 4.15, '1 of 6', fontsize=8, color='royalblue')
plt.text(8.1, 3.2, '2 of 6', fontsize=8, color='royalblue')
plt.text(3.1, 2.2, '4 of 6', fontsize=8, color='royalblue')
plt.text(2.5, 1.2, '6 of 6', fontsize=8, color='royalblue')
plt.text(2.1, 0.18, '6 of 6', fontsize=8, color='royalblue')
plt.tight_layout()
None
```

Performance Overview
Weighted performance index | relative rank
Price
1 of 6
Convenience
2 of 6
Relationship
4 of 6
Service
6 of 6
Selection
6 of 6
Competitor E
Competitor D
Competitor C
Competitor B
Competitor A
Our Business